Don't Crash!! — Development Report

Unit: 6G4Z0020 Programming | Assessment 2 (Cwk 70%) | Target: 80%+
Game theme: Star Wars / asteroid field (player fixed at centre)
Student name: _____ JEI number: _____

1. How the solution was developed

I developed Don't Crash!! in Processing one feature at a time, testing after each step.

I started with DontCrash.pde: a window, starfield, and three states (start, playing, game over). Next I added Player, always drawn at the screen centre, and a basic Obstacle alien drawn with shapes.

I then implemented the required control model. Arrow keys do not move the ship; applyArrowPush() shifts every world object and the starfield in the opposite direction using moveBy(). Hold flags in keyPressed/keyReleased keep steering responsive, and the ship faces the last arrow pressed.

Collision came next. Player.checkCollision(Obstacle) returns a boolean using dist(). A hit loses a life, flashes the screen, starts invulnerability frames, and spawns an explosion. At zero lives the run ends and the high score is saved.

I replaced fixed aliens with ArrayList<Obstacle> aliens. spawnAlienAtEdge() creates enemies over time; objects are marked finished first and removed once at the end of the frame. Obstacles use PImage sprites. FastObstacle and MovingObstacle extend Obstacle and override update/display, so the list holds mixed types. HealthPack pickups restore lives up to three. SPACE fires Bullet lasers along the facing direction.

For higher bands I added PImage animation sequences (thruster flames, alien frames, explosion frames), ScoreManager file I/O with loadStrings/saveStrings on data/highscore.txt, and score-based levels. Finally I introduced the GameEntity interface and abstract WorldObject so most entities share update/display/moveBy, and Explosion plays while gameplay continues. I also refactored into helper methods and private fields for maintainability.

2. Where marking features appear in the code

Band	Feature	Location (tab / method)
40%	Player stays centred; opposite arrow shift	Player constructor; DontCrash.applyArrowPush()
40–50%	2 classes; boolean collision; ≥2 obstacles	Player.pde / Obstacle.pde; Player.checkCollision(Obstacle); restartGame()
50–60%	Own movement; PImage; ArrayList; spawn/remove	Obstacle.update(); loadImage; aliens ArrayList; spawnAlienAtEdge(); removeDeadAliens()
60–70%	2nd obstacle type + inheritance; facing art	FastObstacle.pde, MovingObstacle.pde; Player.currentSprite()
70–80%	PImage sequences; polymorphism; file high score; refactor	flame_/alien_anim_/ explosion_ frames; ArrayList polymorphism; ScoreManager.loadHighScore()/saveHighScore(); drawPlaying helpers
80%+	Entity polymorphism; interface/abstract; collision animation while playing	GameEntity.pde; WorldObject.pde; Explosion.pde + effects ArrayList

3. Code snippet screenshots from development stages

Fig.1 — Boolean collision method (Player.pde)

Snippet A — Player.pde boolean collision (40–50%)

```
88|   }
89|
90|   int getFacing() {
91|     return facing;
92|   }
93|
94|   float getCollisionRadius() {
95|     return bodySize * 0.45;
96|   }
97|
98|   boolean checkCollision(Obstacle obstacle) {
```

Fig.2 — Opposite world push (DontCrash.applyArrowPush)

Snippet B — DontCrash.pde opposite arrow push (40%)

```
155|     starY[i] = starY[i] + dy;
156|     if (starX[i] < 0) starX[i] = starX[i] + width;
157|     if (starX[i] >= width) starX[i] = starX[i] - width;
158|     if (starY[i] < 0) starY[i] = starY[i] + height;
159|     if (starY[i] >= height) starY[i] = starY[i] - height;
160|   }
161| }
162|
163| // Push the whole world opposite to the held arrow keys
164| void applyArrowPush() {
165|   float dx = 0;
166|   float dy = 0;
167|
168|   if (holdLeft)  dx = dx + pushSpeed;
169|   if (holdRight) dx = dx - pushSpeed;
170|   if (holdUp)    dy = dy + pushSpeed;
171|   if (holdDown)  dy = dy - pushSpeed;
172|
173|   syncShipFacingToKeys();
174|
175|   if (dx == 0 && dy == 0) return;
```

Fig.3 — ArrayList and spawning

Snippet C — ArrayList + spawn (50–60%)

```
 16| ArrayList<Obstacle> aliens;
 17| ArrayList<HealthPack> healthPacks;
 18| ArrayList<GameEntity> effects;
 19| ArrayList<Bullet> bullets;
 20|
 21| int fireCooldown = 0;
 22| final int FIRE_COOLDOWN_FRAMES = 12;
...
448|       showHitMessage("Lives Full (3/3)", 180, 220, 180, 55);
449|     }
450|     pack.markDead();
451|   }
452| }
453|
454| void spawnAlienAtEdge() {
455|   int maxAliens = MAX_ALIENS;
456|   if (scores.getLevel() >= 3) {
457|     maxAliens = MAX_ALIENS + 2;
458|   }
459|   if (aliens.size() >= maxAliens) return;
```

Fig.4 — Inheritance (FastObstacle)

Snippet D — FastObstacle inheritance (60–70%)

```
/**
 * FastObstacle.pde — Fast alien subclass with stronger centre pull and dashes.
 * Extends Obstacle; overrides update/display for aggression and a speed streak.
 */
class FastObstacle extends Obstacle {
  int dashCooldown;

  FastObstacle(float startX, float startY, float obstacleSize, PImage sprite) {
    super(startX, startY, obstacleSize, random(3.5, 5.0), color(255, 150, 40), sprite);
    dashCooldown = int(random(40, 100));
  }

  @Override
  void update() {
    if (random(1) < 0.08) {
      float cx = width / 2.0;
```

Fig.5 — High-score file handling (ScoreManager)

Snippet E — ScoreManager file I/O (70–80%)

```
  void loadHighScore() {
    String[] lines = loadStrings(HIGH_SCORE_FILE);
    if (lines != null && lines.length > 0) {
      highScore = int(lines[0]);
      if (highScore < 0) {
        highScore = 0;
      }
      println("Loaded high score: " + highScore);
    } else {
      highScore = 0;
      println("No high score file yet — starting at 0");
    }
  }

  void saveHighScore() {
    if (currentScore > highScore) {
      highScore = currentScore;
      println("New high score: " + highScore);
    }
    String[] data = new String[1];
    data[0] = str(highScore);
    saveStrings(HIGH_SCORE_FILE, data);
  }

  void resetRun() {
```

Fig.6 — Interface and abstract class

Snippet F — Interface + abstract class (80%+)

```
/**
 * GameEntity.pde — Shared interface for updateable / drawable game objects.
 * Enables polymorphism via ArrayList<GameEntity>.
 */
interface GameEntity {
  void update();
  void display();
  void moveBy(float dx, float dy);

  // True when the object should be removed from its list
  boolean isFinished();
}
```

```
/**
 * WorldObject.pde — Abstract base for objects that exist in world space.
 * Stores position/size and shared helpers used by aliens, pickups, bullets, VFX.
 */
abstract class WorldObject implements GameEntity {
  float x;
  float y;
  float size;

  WorldObject(float startX, float startY, float objectSize) {
```

Fig.7 — Explosion animation sequence

Snippet G — Explosion PImage sequence while game continues (80%+)

```
/**
 * Explosion.pde — Collision animation that plays while the game continues.
 * Cycles a PImage frame sequence (80%+ / 70–80% animation criterion).
 */
class Explosion extends WorldObject {
  int life;
  final int MAX_LIFE = 30;
  PImage[] frames;
  int frameIndex;
  int frameTimer;
  final int FRAME_HOLD = 6;

  Explosion(float cx, float cy, PImage[] animFrames) {
    super(cx, cy, 64);
    life = MAX_LIFE;
    frames = animFrames;
    frameIndex = 0;
    frameTimer = 0;
  }

  void update() {
    life--;
    frameTimer++;
    if (frames != null && frameTimer >= FRAME_HOLD) {
      frameTimer = 0;
      if (frameIndex < frames.length - 1) {
        frameIndex = frameIndex + 1;
      }
```

Development narrative word count: 240 (limit 500). Replace the name/JEI fields before Moodle submission.

Note: Review this report so you can explain every step in a viva. Add any personal debugging stories if asked.